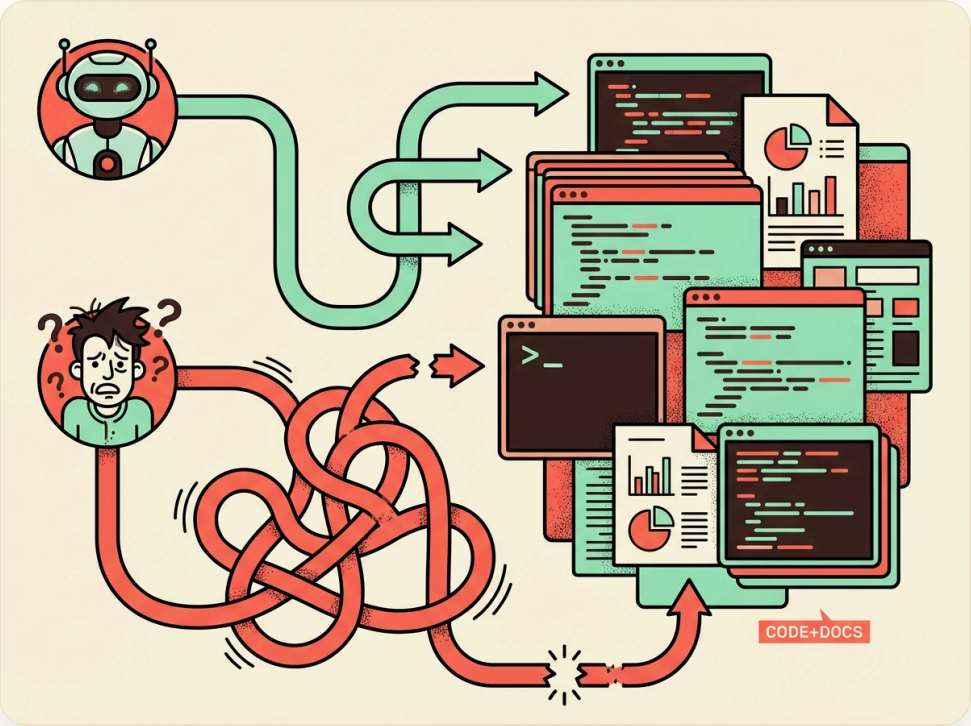
AIフレンドリーなコードは
なぜ人間には重いのか

vibe coding時代の保守運用メモ



今日の結論

- AI中心で作られたコードは、整っていても人間には重くなりやすい
- 原因は「AIが必要とする明示情報量」と「人の暗黙知」の差
- 対策は、AI中心リポジトリをAI経由で読む/直す運用に寄せること

きっかけ

- 社内の小規模ツールを軽微改修
- コード品質もドキュメント品質も高かった
- ただし変更の体感コストが高かった
  - docstring などがきっちり書かれているため、少しの変更でも修正箇所が広がる
  - ドキュメント量が多く、仕様を読み切ること自体に大きな負荷がかかる

起きていたこと

- プロジェクト規模に対して、コードと文書の情報量が多い
- docstring / 設計説明 / 補足文書の整合を保つコストが高い
- 小さな変更でも更新範囲が広がりやすい
- 品質の問題ではなく、認知負荷の問題

なぜズレが起きるか

観点	人間	AI
前提	背景知識を使う	暗黙知を使えない
書き方	省略しがち	明示情報を厚く書く
結果	必要最低限で速く読める	やや冗長なdocumentやコード

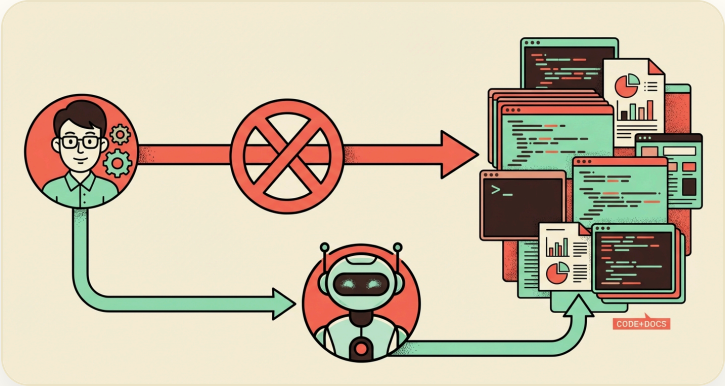
この差が「AIフレンドリー」と「Humanフレンドリー」のズレになる

新しいボトルネック

- 実装速度は上がる
- 一方で読解とレビューが詰まりやすくなる
- 問題は「AI導入で速くなった」だけではない
- 情報密度のミスマッチ自体が負荷を増やす

運用提案

- AI中心で作られたリポジトリは、基本AI経由で扱う
  - 実装
  - 仕様確認
  - 影響範囲把握
- README冒頭に運用方針を明記する
  - `AI経由での改修を推奨`



レビュー方針の再設計

- 細部品質はAI + 実装者を一定信頼する
- レビュワーは以下に集中する
  - インターフェース変更
  - 外部結合点への影響
  - チーム全体の仕様整合

信頼度の線引き

- 失敗コストが低い領域
  - AI実装を厚めに信頼
  - レビューは仕様中心に軽量化
- 失敗コストが高い領域
  - 人間レビューを厚く維持
  - 詳細コードも従来通り確認

まとめ

- vibe codings産コードは AIフレンドリー⭕ Humanアンフレンドリー❌
- AIで作ったコードは、全てAIを通じて読み書きする
- レポジトリごとにAIへの信用レベルをチームで合意する